



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	ONG-IKO, KATELYN M.
Section:	AN43
Professor:	MS CRISOLA TAN
Date:	10/9/26

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements

- Loops (for loop)
 - Functions / methods
2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... × + v
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks

TASK 1: SET UP THE ENVIRONMENT

Ask the user for the initial state of each room

```
rooms = {}
```

```
for room in ["A", "B"]:
```

```
    while True:
```

```
        state = input(f'Is Room {room} dirty or clean? ').strip().lower()
```

```

    if state in ["dirty", "clean"]:
        rooms[room] = state
        break
    else:
        print("Please enter only 'dirty' or 'clean'.")

# Function to check if a room is dirty
def is_dirty(room):
    return rooms[room] == "dirty"

# Function to clean a room
def clean_room(room):
    rooms[room] = "clean"

```

```

print("\nInitial Environment:", rooms)
*** Is Room A dirty or clean? clean
    Is Room B dirty or clean? dirty

    Initial Environment: {'A': 'clean', 'B': 'dirty'}

```

TASK 2: IMPLEMENT THE RULE-BASED AGENT

```

class VacuumAgent:
    def __init__(self, location="A"):
        self.location = location

    # Move to the other room
    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    # Decide what action to take
    def perceive_and_act(self):
        if is_dirty(self.location):
            clean_room(self.location)
            action = "Cleaned Room " + self.location
        else:
            self.move()
            action = "Moved to Room " + self.location

    return action

```

```
# Create the agent starting in Room A
agent = VacuumAgent("A")

print("Agent created at Room", agent.location)
```

```
... Agent created at Room A
```

```
# TASK 3: RUN THE SIMULATION
```

```
while True:
    try:
        steps = int(input("Enter the number of simulation steps: "))
        if steps > 0:
            break
        print("Please enter a number greater than zero.")
    except ValueError:
        print("Please enter a valid whole number.")

print("\n--- SIMULATION START ---")

for step in range(1, steps + 1):
    action = agent.perceive_and_act()

    print(f"\nStep {step}")
    print("Agent Location:", agent.location)
    print("Action Taken:", action)
    print("Room Conditions:", rooms)

print("\n--- SIMULATION COMPLETE ---")
print("Final Environment:", rooms)
```

```
Enter the number of simulation steps: 2

--- SIMULATION START ---

Step 1
Agent Location: B
Action Taken: Moved to Room B
Room Conditions: {'A': 'clean', 'B': 'dirty'}

Step 2
Agent Location: B
Action Taken: Cleaned Room B
Room Conditions: {'A': 'clean', 'B': 'clean'}

--- SIMULATION COMPLETE ---
Final Environment: {'A': 'clean', 'B': 'clean'}
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. What rules does the agent follow?

The agent follows two rules: if its current room is dirty, it cleans the room. If the room is already clean, it moves to the other room.

2. How does the agent decide when to clean and when to move?

The agent checks the condition of its current room. It cleans the room if it is dirty and moves to the other room if it is clean.

3. What happens if both rooms are already clean at the start?

The agent will move between Rooms A and B without cleaning anything because both rooms are already clean.

4. How would you modify the agent to handle more than two rooms?

I would store the rooms in a list or dictionary and modify the movement method to select the next room. The agent would check and clean each dirty room until all rooms are clean.